Code API

This appendix documents the public C++ API for continuous-time state estimation using Gaussian Process interpolation under a White-Noise-on-Acceleration (WNOA) motion prior. All classes and functions reside in the `gtsam` namespace.

StateData

Header: `gtsam/nonlinear/StateData.h`

Lightweight container for states used for continuous-time estimation and interpolation. `StateData` stores the keys identifying the pose and velocity variables in a factor graph together with the associated timestamp. It is intentionally minimal and used primarily for ordering, lookup and grouping of states when constructing interpolation queries.

Members

`Key pose`
Key of the pose variable.

`Key vel`
Key of the velocity variable.

`double time`
Timestamp (seconds) associated with this state.

Constructors

`StateData()`
Default constructor (value-initialized members). Provided for convenience and to enable use in standard containers.

`StateData(Key pose_in, Key vel_in, double time_in)`
Construct a `StateData` from explicit keys and timestamp.

- `pose_in`: Key for the pose variable.
- `vel_in`: Key for the velocity variable.
- `time_in`: Timestamp in seconds.

Operators

`bool operator<(const StateData& other) const`
Compare by timestamp (ascending). Useful to sort state lists in chronological order.

- `other`: StateData to compare against.
- *Returns*: `true` if `this->time < other.time`.

`bool operator>(const StateData& other) const`
Compare by timestamp (descending).

- `other`: StateData to compare against.
- *Returns*: `true` if `this->time > other.time`.

`bool operator<(double time) const`
Compare this state's timestamp with a raw time value.

- `time`: Timestamp to compare against.
- *Returns*: `true` if `this->time < time`.

`bool operator>(double time) const`
Compare this state's timestamp with a raw time value.

- `time`: Timestamp to compare against.
- *Returns*: `true` if `this->time > time`.

`bool operator==(const StateData& other) const`

Equality compares the identifying keys (pose and vel). Two `StateData` objects are considered equal if they refer to the same pose and velocity keys. The timestamp is not used for equality to keep `StateData` usable as a simple identifier in unordered containers.

- `other`: StateData to compare against.
- *Returns*: `true` if both pose and vel keys match.

Hash Support

A `std::hash<StateData>` specialization combines the hash of the pose and velocity keys to produce a stable hash suitable for `unordered_map`/`unordered_set`. The timestamp is not included because `StateData` equality ignores time.

Interpolator

Header: `gtsam/nonlinear/Interpolator.h`

The `Interpolator<PoseType>` class provides routines to interpolate pose and velocity at arbitrary times between (or outside) estimated states. It encapsulates the motion model (WNOA by default) via function objects for transition, covariance and Jacobian computations. The class also computes conditional covariances for interpolated states when requested.

Template Parameter

`PoseType`
Pose group/type (e.g. `Pose2`, `Pose3`, or a vector-space pose).

Type Aliases

`StateDataInterval`
`std::pair<std::optional<StateData>, std::optional<StateData>>`. Pair of optional bordering states for an interpolated state. Either entry may be empty (`std::nullopt`) to indicate a missing border; in that case extrapolation is used instead of interpolation.

`PoseVelocity<PoseType>`
Simple container for a pose and its corresponding velocity. Used as the local representation of a state when interpolating pose and velocity pairs.

`TimestampedPoseVelocity<PoseType>`
Wraps a `PoseVelocity` together with a timestamp. This is the primary representation passed to interpolation routines.

`CovarianceMap`
`std::map<Key, Matrix>`. Maps a pose or velocity key to its covariance matrix.

`LambdaPsiMats`
`std::pair<Matrix2N, Matrix2N>`. Pair of $(\mathbf{\Lambda}, \mathbf{\Psi})$ interpolation matrices.

`LocalStateVecs`
`std::pair<VectorN, VectorN>`. Pair of local state vectors $(\boldsymbol{\xi}, \dot{\boldsymbol{\xi}})$.

`LocalGlobalStateJacs`
`std::vector<MatrixN>`. Vector of local-to-global Jacobian matrices.

Constructors

`Interpolator(const VectorN& Q_psd, bool small_angle_approx = false)`
Default constructor using the WNOA motion model.

- `Q_psd`: Diagonal power spectral density vector for the motion prior.
- `small_angle_approx`: If true, use small-angle approximations on Lie groups.

`Interpolator(const VectorN& Q_psd, bool small_angle_approx,`
`std::function<Matrix(double)> transitionFunction,`
`std::function<Matrix(double, const VectorN&)> covarianceFunction,`
`std::function<Matrix(double, const VectorN&)> inverseCovarianceFunction,`
`std::function<Matrix(const PosePair&, const PosePair&, double)>`

```
computeJacobianPrev,
std::function<Matrix(const PosePair&, const PosePair&, double)>
computeJacobianNext)
```

Construct an Interpolator with custom motion-model functions.

- `Q_psd`: Diagonal power spectral density vector for the motion prior.
- `small_angle_approx`: If true, use small-angle approximations on Lie groups.
- `transitionFunction`: Function returning the transition matrix for a given Δt.
- `covarianceFunction`: Function returning process noise covariance.
- `inverseCovarianceFunction`: Function returning inverse covariance.
- `computeJacobianPrev`: Function computing the Jacobian w.r.t. the previous bordering state.
- `computeJacobianNext`: Function computing the Jacobian w.r.t. the next bordering state.

Public Methods

```
PoseVel interpolatePoseAndVelocity(
    const std::optional<TimestampedPoseVel>& Tvarpi_k,
    const std::optional<TimestampedPoseVel>& Tvarpi_kp1,
    double t_tau,
    OptionalMatrixVecType H = nullptr,
    const std::shared_ptr<Matrix>& mainSolveMarginalMatrix = nullptr,
    Matrix* covarianceOut = nullptr,
    const std::shared_ptr<const LambdaPsiMats>& LambdaPsiPreComp = nullptr,
    const std::shared_ptr<const LocalStateVecs>& localStateVecsPreComp =
    nullptr,
    const std::shared_ptr<const LocalGlobalStateJacs>& localGlobalStateJacsPreComp
    = nullptr) const
```

Interpolate the pose and velocity at time t_τ.

- `Tvarpi_k`: Optional timestamped pose/velocity at left border t_k.
- `Tvarpi_kp1`: Optional timestamped pose/velocity at right border t_{k+1}.
- `t_tau`: Interpolation time τ.
- `H`: Optional output vector of Jacobian blocks (8 matrices).
- `mainSolveMarginalMatrix`: Optional marginal covariance of bordering states from the main solve.
- `covarianceOut`: Optional output pointer for the interpolated state's covariance.
- `LambdaPsiPreComp`: Optional precomputed $\mathbf{\Lambda}/\mathbf{\Psi}$ matrices.
- `localStateVecsPreComp`: Optional precomputed local state vectors $(\boldsymbol{\xi}, \dot{\boldsymbol{\xi}})$.
- `localGlobalStateJacsPreComp`: Optional precomputed local→global Jacobians.
- *Returns*: Interpolated pose and velocity pair at t_τ.

```
Values interpolatePosesAndVelocities(
    const NonlinearFactorGraph& mainSolveGraph,
    const Values& mainSolveSolution,
    const StateDataSet& mainSolveStates,
    const StateDataSet& interpolatedStates,
    std::shared_ptr<CovarianceMap> covarianceMapOut = nullptr) const
```

Interpolate multiple poses and velocities given a solved factor graph.

- `mainSolveGraph`: Factor graph used in the main solve (estimated states only).
- `mainSolveSolution`: Solution `Values` from the main solve.
- `mainSolveStates`: Ordered set of estimated `StateData` used in the main solve.
- `interpolatedStates`: Set of `StateData` to interpolate.
- `covarianceMapOut`: Optional output map to receive per-state covariances.
- *Returns*: A `Values` container with interpolated pose and velocity entries.

```
Matrix2N computeConditionalCov(
    const TimestampedPoseVel& pvk,
    const TimestampedPoseVel& pvkp1,
    const TimestampedPoseVel& pvtau,
    OptionalMatrixType Lambda = nullptr,
    OptionalMatrixType Psi = nullptr) const
```

Compute the conditional covariance of the interpolated state.

- `pvk`: Timestamped pose/velocity at left border.
- `pvkp1`: Timestamped pose/velocity at right border.
- `pvtau`: Timestamped pose/velocity at interpolation time.
- `Lambda`: Optional output pointer to receive $\mathbf{\Lambda}$ matrix.
- `Psi`: Optional output pointer to receive $\mathbf{\Psi}$ matrix.
- *Returns*: Conditional covariance matrix $\mathbf{\Sigma}_\tau$ of size $2N \times 2N$.

```
std::pair<Matrix, Matrix> getLambdaPsi(double t_k, double t_kp1, double
    t_tau) const
```

Compute interpolation matrices $\mathbf{\Lambda}$ and $\mathbf{\Psi}$ (WNOA-optimized).

- `t_k`: Time of the left border state.
- `t_kp1`: Time of the right border state.
- `t_tau`: Interpolation time.
- *Returns*: Pair $(\mathbf{\Lambda}, \mathbf{\Psi})$ of interpolation matrices.

```
std::pair<Matrix, Matrix> getLambdaPsiGeneral(double t_k, double t_kp1,
    double t_tau) const
```

Compute interpolation matrices $\mathbf{\Lambda}$ and $\mathbf{\Psi}$ for a general motion model.

- `t_k`: Time of the left border state.
- `t_kp1`: Time of the right border state.
- `t_tau`: Interpolation time.
- *Returns*: Pair $(\mathbf{\Lambda}, \mathbf{\Psi})$ of interpolation matrices.

```
LocalStateVecs computeLocalStateVecs(
    const TimestampedPoseVel& pvk,
    const TimestampedPoseVel& pvkp1,
    LocalGlobalStateJacs* jacs = nullptr) const
```

Compute the local state vectors $(\boldsymbol{\xi}, \dot{\boldsymbol{\xi}})$ for a pair of bordering states.

- `pvk`: Timestamped pose/velocity at left border state.
- `pvkp1`: Timestamped pose/velocity at right border state.
- `jacs`: Optional output pointer to receive local-to-global Jacobians.
- *Returns*: Pair of local state vectors $(\boldsymbol{\xi}, \dot{\boldsymbol{\xi}})$.

WNOAMotionFactor

Header: `gtsam/nonlinear/WNOAFactor.h`

WNOA (White Noise on Acceleration) motion prior factor. This factor implements the WNOA motion prior between two states (pose and velocity at times t_k and t_{k+1}). It provides residuals and Jacobians consistent with the WNOA continuous-time prior when discretized over the timestep `delta_t`.

Template parameter `Pose` is expected to be a GTSAM pose type (e.g. `Pose2`, `Pose3`) or a vector-space pose; the factor supports both Lie group and vector-space pose representations. The factor's ordering of keys is: `pose_k`, `vel_k`, `pose_kp1`, `vel_kp1`.

Template Parameter

`Pose`

Pose group/type (`Pose2`, `Pose3`, etc.)

Constructors

`WNOAMotionFactor(const StateData& state_k, const StateData& state_kp1, const VectorN& Q)`

Construct a WNOA motion factor from two `StateData` entries.

- `state_k`: State data at time t_k (contains pose key, vel key, and timestamp).
- `state_kp1`: State data at time t_{k+1}.
- `Q`: Diagonal power spectral density vector for the WNOA motion prior.

Builds a factor connecting the pose and velocity keys contained in `state_k` and `state_kp1`. The internal noise model is constructed from the provided diagonal PSD vector `Q` and the timestep computed from the two state timestamps.

`WNOAMotionFactor(Key key1, Key key2, Key key3, Key key4, double delta_t, const VectorN& Q)`

Construct a WNOA factor given explicit keys and timestep.

- `key1`: Key of the pose at time t_k.
- `key2`: Key of the velocity at time t_k.
- `key3`: Key of the pose at time t_{k+1}.
- `key4`: Key of the velocity at time t_{k+1}.
- `delta_t`: Time difference $\Delta t = t_{k+1} - t_k$.
- `Q`: Diagonal power spectral density vector for the WNOA motion prior.

Public Methods

```
Vector evaluateError(
    const Pose& pose_k, const typename Pose::TangentVector& vel_k,
    const Pose& pose_kp1, const typename Pose::TangentVector& vel_kp1,
    OptionalMatrixType H1 = nullptr, OptionalMatrixType H2 = nullptr,
    OptionalMatrixType H3 = nullptr, OptionalMatrixType H4 = nullptr) const
    override
```

Evaluate the WNOA factor residual and optional Jacobians.

- `pose_k`: Pose at time t_k.
- `vel_k`: Velocity at time t_k.

- `pose_kp1`: Pose at time t_{k+1}.
- `vel_kp1`: Velocity at time t_{k+1}.
- `H1`: Optional output Jacobian w.r.t. `pose_k`.
- `H2`: Optional output Jacobian w.r.t. `vel_k`.
- `H3`: Optional output Jacobian w.r.t. `pose_kp1`.
- `H4`: Optional output Jacobian w.r.t. `vel_kp1`.
- *Returns*: Residual vector $\mathbf{e} \in \mathbb{R}^{2N}$ formed from the difference between the discrete relative pose/velocity and the predicted motion under the WNOA prior.

Static Methods

`static Matrix2N buildWNOACovariance(double timestep, const VectorN& Q)`

Build the continuous-time WNOA discretized process covariance.

- `timestep`: The time interval Δt over which to discretize.
- `Q`: Diagonal power spectral density vector.
- *Returns*: The $2N \times 2N$ covariance matrix for a WNOA prior discretized over `timestep`.

`static Matrix2N buildInverseWNOACovariance(double timestep, const VectorN& Q)`

Build the inverse of the WNOA discretized process covariance.

- `timestep`: The time interval Δt over which to discretize.
- `Q`: Diagonal power spectral density vector.
- *Returns*: The $2N \times 2N$ inverse covariance (information) matrix.

`static noiseModel::Gaussian::shared_ptr buildWNOANoiseModel(double timestep, const VectorN& Q)`

Convenience helper to construct a Gaussian noise model from `Q`.

- `timestep`: The time interval Δt.
- `Q`: Diagonal power spectral density vector.
- *Returns*: Shared pointer to a Gaussian noise model.

`static Matrix2N transitionFunction(double delta_t)`

Transition matrix for the WNOA prior.

- `delta_t`: Time interval Δt.
- *Returns*: The $2N \times 2N$ transition matrix mapping for concatenated state $[\text{pose}; \text{vel}]$ over interval `delta_t`.

`static Matrix2N computeJacobianPrev(const PosePair& pose_pair_k, const PosePair& pose_pair_tau, double t_diff)`

Compute interpolation Jacobian with respect to the previous state.

- `pose_pair_k`: Pose/velocity pair at the left border t_k.
- `pose_pair_tau`: Pose/velocity pair at the interpolation time τ.
- `t_diff`: Time difference $\tau - t_k$.
- *Returns*: The $2N \times 2N$ Jacobian block that maps perturbations in the previous bordering state to perturbations in the interpolated discrete residual.

`static Matrix2N computeJacobianNext(const PosePair& pose_pair_kp1, const PosePair& pose_pair_tau, double t_diff)`

Compute interpolation Jacobian with respect to the next state.

- `pose_pair_kp1`: Pose/velocity pair at the right border t_{k+1}.

- `pose_pair_tau`: Pose/velocity pair at the interpolation time τ.
- `t_diff`: Time difference $t_{k+1} - \tau$.
- *Returns*: The $2N \times 2N$ Jacobian block that maps perturbations in the next bordering state to perturbations in the interpolated discrete residual.

WNOAInterpFactor

Header: `gtsam/nonlinear/WNOAInterpFactor.h`

Wrapper class that allows a variable of a factor to be replaced by a WNOA interpolation. It is assumed that estimated state entries are sorted by time. The wrapper maps an inner factor that references "interpolated" states onto the outer estimated states used by the main solve.

Template Parameter

`PoseType`
: Pose group/type (e.g. `Pose2`, `Pose3`).

Nested Types

`PassedInterpData`
: Container for externally-computed interpolation data. Callers can precompute interpolated `Values`, flattened inner→outer jacobian blocks and conditional covariances and pass them into `linearize`/`error` to avoid recomputation inside the factor.

- `Values values`: Interpolated Values (pose and velocity entries at interpolated times).
- `unordered_map<Key, std::array<Matrix, 4> jacobians`: Flattened jacobian blocks for each inner key; array order is [LPose, LVel, RPose, RVel].
- `unordered_map<StateData, Matrix2N> condCovs`: Conditional covariance ($2N \times 2N$) for each interpolated StateData.

Constructor

```
WNOAInterpFactor(
    NonlinearFactor::shared_ptr inner_factor,
    const StateDataSet& estimated_states,
    const StateDataSet& interp_states,
    const VectorN& Q_psd,
    bool fixed_noise_model = false,
    bool precomp_interp_mats = true,
    bool small_angle_approx = false)
```

Construct a WNOA interpolation wrapper factor.

- `inner_factor`: Factor defined over (possibly) interpolated keys.
- `estimated_states`: Ordered set of estimated `StateData` (main-solve states).
- `interp_states`: Ordered set of states to interpolate (states at query times).
- `Q_psd`: Diagonal PSD vector for the WNOA motion prior.
- `fixed_noise_model`: If true, do not augment the measurement noise model with GP interpolation process noise.
- `precomp_interp_mats`: If true, precompute $\mathbf{\Lambda}/\mathbf{\Psi}$ matrices for each interpolated state.
- `small_angle_approx`: If true, enable the small-angle approximation in interpolation.

Public Methods

`Vector unwhitenedError(const Values& values, OptionalMatrixVecType H = nullptr, PassedInterpData* passedInterpData = nullptr) const override`

Compute the unwhitened residual vector for the wrapped inner factor.

- `values`: Current estimated `Values` (outer/main-solve states).
- `H`: Optional output vector for Jacobian blocks.
- `passedInterpData`: Optional precomputed interpolation data to avoid duplicate work.
- *Returns*: The outer-facing residual (before whitening) obtained by evaluating the inner factor on interpolated states and mapping the resulting error back to the outer state ordering.

`boost::shared_ptr<GaussianFactor> linearize(const Values& x) const override`

Linearize the wrapper factor, computing a `JacobianFactor`.

- `x`: Linearization point (estimated state `Values`).
- *Returns*: Shared pointer to linearized `GaussianFactor`. When interpolation increases measurement uncertainty, this function updates the noise model on-the-fly.

`boost::shared_ptr<GaussianFactor> linearize(const Values& x, PassedInterpData* passedInterpData) const`

Linearize using externally provided interpolation data to avoid duplicate work.

- `x`: Linearization point (estimated state `Values`).
- `passedInterpData`: Precomputed interpolation data containing values, jacobians, and covariances.
- *Returns*: Shared pointer to linearized `GaussianFactor`.

`double error(const Values& c) const override`

Compute the factor error (unwhitened) using the current `Values`.

- `c`: Current estimated `Values`.
- *Returns*: Scalar error value.

`double error(const Values& c, PassedInterpData* passedInterpData) const`

Error computation accepting precomputed interpolation data.

- `c`: Current estimated `Values`.
- `passedInterpData`: Precomputed interpolation data.
- *Returns*: Scalar error value.

`SharedGaussian noiseModel(const Values& x) const`

Return an augmented noise model accounting for interpolation.

- `x`: Current estimated `Values`.
- *Returns*: Shared pointer to Gaussian noise model (possibly augmented with interpolation uncertainty).

`std::unordered_map<Key, StateData> getInterpolatedKeys() const`

Get the mapping from outer Keys to their corresponding interpolated `StateData`.

- *Returns*: Map from each inner factor key (that requires interpolation) to its `StateData`.

`std::unordered_map<StateData, std::pair<StateData, StateData>> getInterpToBorders() const`

Get the mapping from each interpolated `StateData` to its bordering states.

- *Returns*: Map from each interpolated state to its (left, right) bordering estimated states.

WNOAFactorGraph

Header: `gtsam/nonlinear/WNOAFactorGraph.h`

Factor graph specialized for WNOA interpolation-aware computation. `WNOAFactorGraph` wraps standard factor graph functionalities with utilities to compute interpolated pose/velocity states under a White-Noise-on-Acceleration (WNOA) motion prior. It stores the mapping from interpolated query times to their left/right bordering estimated states and precomputes interpolation helpers for efficient repeated evaluation.

The graph provides optimized linearization and error computation routines that can exploit precomputed interpolation batches to reduce repeated work when evaluating many wrapper factors using the same query times.

Template Parameter

`PoseType`

Pose group/type (e.g. `Pose2`, `Pose3`) used by the interpolator.

Constructor

```
WNOAFactorGraph(
    std::unordered_map<StateData, std::pair<StateData, StateData>>
    interp_map,
    const VectorN& Q_psd,
    bool fixed_noise_model = false)
```

Construct a `WNOAFactorGraph` with interpolation metadata.

- `interp_map`: Mapping from each interpolated `StateData` to its left/right bordering estimated `StateData`.
- `Q_psd`: Diagonal PSD vector for the WNOA interpolator (size must match PoseType dimension).
- `fixed_noise_model`: If true, the graph will not augment measurement noise for interpolation.

Public Methods

```
boost::shared_ptr<GaussianFactorGraph> linearize(const Values&
    linearizationPoint) const
```

Linearize the graph into a `GaussianFactorGraph`.

- `linearizationPoint`: Current estimate of all variables.
- *Returns*: Shared pointer to the linearized Gaussian factor graph. Exploits precomputed interpolation data and batching to reduce duplicated interpolation work across wrapper factors.

```
double error(const Values& values) const
```

Compute the unnormalized graph error (sum of factor losses).

- `values`: Current estimated `Values`.
- *Returns*: Total error (scalar). When the graph contains interpolation wrapper factors this method uses the interpolator to evaluate interpolated states and can exploit precomputation to improve throughput.

WNOALevenbergMarquardtOptimizer

Header: `gtsam/nonlinear/WNOALevenbergMarquardtOptimizer.h`

Levenberg-Marquardt optimizer tuned for WNOA interpolation graphs. `WNOALevenbergMarquardtOptimizer` implements the Levenberg-Marquardt trust-region algorithm and is specialized to operate on `WNOAFactorGraph<PoseType>`. It can leverage interpolation precomputation in the factor graph to reduce redundant work during linearization and error evaluation when many wrapper factors are present.

Template Parameter

`PoseType`

Pose group/type used by the underlying WNOA graph (e.g. `Pose2`, `Pose3`).

Constructors

```
WNOALevenbergMarquardtOptimizer(
    const WNOAFactorGraph<PoseType>& graph,
    const Values& initialValues,
    const LevenbergMarquardtParams& params
    = LevenbergMarquardtParams())
```

Construct optimizer for a `WNOAFactorGraph` with initial values.

- `graph`: The `WNOAFactorGraph` to optimize.
- `initialValues`: Initial variable estimates.
- `params`: LM algorithm parameters (default uses standard settings).

```
WNOALevenbergMarquardtOptimizer(
    const WNOAFactorGraph<PoseType>& graph,
    const Values& initialValues,
    const Ordering& ordering,
    const LevenbergMarquardtParams& params
    = LevenbergMarquardtParams())
```

Construct optimizer with explicit variable ordering for elimination/linearization.

- `graph`: The `WNOAFactorGraph` to optimize.
- `initialValues`: Initial variable estimates.
- `ordering`: Variable elimination ordering.
- `params`: LM algorithm parameters.

Public Methods

Public methods are equivalent to those of the regular `LevenbergMarquardtOptimizer` with the exception that internally, the custom `linearize()` and `error()` functions of the `WNOAFactorGraph` are called directly.

Free Functions

Header: `gtsam/nonlinear/WNOAInterpFactor.h`

interpolateFactorGraph

```
template<typename PoseType>
    NonlinearFactorGraph interpolateFactorGraph(
    const NonlinearFactorGraph& graph,
    const StateDataSet& estimated_states,
    const StateDataSet& interp_states,
    const VectorN& Q_psd,
    bool fixed_noise = false,
    bool small_angle_approx = false)
```

Convert a factor graph by removing interpolated states.

- `graph`: Input factor graph possibly containing factors on interpolated states.
- `estimated_states`: Ordered set of estimated `StateData` (main-solve states).
- `interp_states`: Ordered set of `StateData` entries to be interpolated/removed.
- `Q_psd`: Diagonal PSD vector for the WNOA motion prior.
- `fixed_noise`: If true, do not augment measurement noise models for interpolation.
- `small_angle_approx`: If true, use small-angle approximation for interpolation.
- *Returns*: A new `NonlinearFactorGraph` where factors referencing interpolated states are replaced with equivalent wrapper factors acting on bordering estimated states. WNOA motion prior factors are added between successive estimated states. Factors that do not reference interpolated states are copied unchanged.

interpolateWNOAFactorGraph

```
template<typename PoseType>
    WNOAFactorGraph<PoseType> interpolateWNOAFactorGraph(
    const NonlinearFactorGraph& graph,
    const StateDataSet& estimated_states,
    const StateDataSet& interp_states,
    const VectorN& Q_psd,
    bool fixed_noise = false,
    bool small_angle_approx = false)
```

Convert a factor graph to a `WNOAFactorGraph` with interpolated states removed.

- `graph`: Input factor graph possibly containing factors on interpolated states.
- `estimated_states`: Ordered set of estimated `StateData` (main-solve states).
- `interp_states`: Ordered set of `StateData` entries to be interpolated/removed.
- `Q_psd`: Diagonal PSD vector for the WNOA motion prior.
- `fixed_noise`: If true, do not augment measurement noise models for interpolation.
- `small_angle_approx`: If true, use small-angle approximation for interpolation.
- *Returns*: A `WNOAFactorGraph<PoseType>` where factors on interpolated states are replaced with wrapper factors acting on bordering estimated states. The

returned graph carries interpolation metadata (borders and PSD) for efficient batched evaluation.

updateInterpValues

```
template<typename PoseType>
    Values updateInterpValues(
    const NonlinearFactorGraph& interp_graph,
    const Values& values,
    const StateDataSet& estim_states,
    const StateDataSet& interp_states,
    const VectorN& Q_psd,
    std::shared_ptr<CovarianceMap> covarianceMapOut = nullptr)
```

Update a `Values` with interpolated pose and velocity entries.

- `interp_graph`: Factor graph containing interpolation metadata (borders).
- `values`: Current estimated `Values` (outer states used as borders).
- `estim_states`: Ordered set of estimated `StateData` used by the main solve.
- `interp_states`: Ordered set of `StateData` entries to interpolate.
- `Q_psd`: Diagonal PSD vector for the WNOA motion prior.
- `covarianceMapOut`: Optional output pointer to receive per-interpolated-state covariances.
- *Returns*: A copy of `values` with interpolated pose/velocity entries merged in.